

Tessera AI-as-Indexer — 30-Day Roadmap (v2: On-Prem Inference)

Owner: Matt Hendricks

Target end date: End of May 2026 (30 days from start)

Demo target: Live dashboard processing synthetic insurer mail with $\geq 75\%$ of mails routed at high-confidence tier with $\geq 95\%$ accuracy at that tier — using a model that runs entirely on the customer's own infrastructure with zero external API calls at inference time.

0. What changed from v1, and why

The v1 roadmap put a frontier LLM (Claude API) at the core of Tier 4. That makes the build fast but kills the sale: South African insurers will not approve customer mail, ID numbers, and financial details leaving their perimeter under POPIA, and "we have a DPA with the vendor" is not a compliance argument that survives first contact with a CISO.

v2 fixes this with one architectural shift:

Frontier LLMs are used during development only. The model that runs in production is a small fine-tuned encoder that runs on the customer's own CPU.

Concretely:

- Tier 1 (QR + schema extraction): zero ML, pure deterministic CPU code (unchanged).
- Tier 2 (policy-number regex): zero ML, pure regex (unchanged).
- Tier 3 (fuzzy phrase match): zero ML, TF-IDF with a small SA-specific phrase catalog (unchanged).
- **Tier 4 (the only ML tier):** was Claude Sonnet via API; now a fine-tuned XLM-RoBERTa-base classifier running on customer CPU. $\sim 95\%$ accuracy ceiling, $\sim 200\text{ms}$ inference per email, fits in 1.1GB on disk, deploys as a Docker image with no internet dependency.

Frontier LLMs (Haiku for body generation, Sonnet as eval ceiling) only touch *synthetic* data we generate ourselves. No real customer mail ever leaves the customer's network, ever.

This is also a stronger pitch story because it converts "we wrap GPT" into "we ship you a model trained on synthetic insurance data, runs in your DC, full audit, your data never leaves." That story works at any regulated SA buyer — banks, medical aids, government — not just insurers.

1. Outcome and success criteria

The demo at end of month answers one investor question: *"How much of this can run on the customer's own machine without sending anything anywhere?"*

Headline metric: % of emails routed at high-confidence tier with measured accuracy at that tier, using only on-prem compute.

Demo target (stretch): 75% auto-route at ≥95% precision, on-prem only.

Demo floor (acceptable): 60% auto-route at ≥95% precision, on-prem only.

Comparison ceiling (development tool only): measure how close on-prem performance gets to a Claude Sonnet zero-shot baseline. Target: within 3 points. Frontier LLM is the eval ceiling, not the production model.

Metric	Target	Why it matters
Tier-1 (QR) accuracy	≥99%	Proves deterministic path
Tier-2 (policy number) accuracy	≥97%	Proves legacy-form value without QR rollout
Tier-4 (encoder) accuracy at ≥0.9 confidence	≥95%	The compliance-friendly model works
Calibration error (ECE)	<0.05	Confidence numbers are honest
External API calls at inference time	0	Compliance story is true, not aspirational
Avg latency per email (CPU only)	≤500ms	Realistic for production ops
Model size on disk	≤1.5GB	Customer IT can deploy without negotiating

2. Stack decisions (locked)

Layer	Choice	Rationale
Language	Python 3.11	Standard
Form rendering	ReportLab	Real PDFs, scriptable
QR	<code>qrcode</code> , error correction M	Robust to scanning
OCR	<code>pytesseract</code> (local)	No cloud dependency for production. Document AI optional for benchmarking only.
Tickbox detection	OpenCV custom	Cheaper and more reliable than ML
Body generator (synthetic data only)	Claude Haiku 4.5 via API	Touches synthetic data only, never customer data
Eval ceiling (development reference)	Claude Sonnet 4.6 via API	Used to set the target accuracy for the on-prem model

Layer	Choice	Rationale
only)		
Production classifier	Fine-tuned XLM-RoBERTa-base	Multilingual (handles Afrikaans), 278M params, runs on CPU, deploys offline
Baseline classifier (ablation)	TF-IDF + Logistic Regression	Sub-1MB model, fully explainable, sets the floor
Training environment	Google Colab Free (T4 GPU) or local Mac M-series via MPS	XLM-R base fits in 12GB
Inference framework	Hugging Face Transformers (dev), ONNX Runtime (production)	ONNX gives 2-3x CPU speedup
Eval framework	Custom (no external tool)	Transparent, ships with the model
Audit DB	SQLite for dev, Postgres for demo	Both run on-prem
Dashboard	Next.js + shadcn + Recharts	Polish matters
Deployment	Docker image, no internet required at runtime	Compliance-first

3. Repository structure

```
tessera-indexer/  
├─ specs/                                # SDD specs handed to dev agents  
├─ taxonomy/  
│   ├─ taxonomy.yaml  
│   └─ schemas/                          # 6 form schemas  
├─ generator/                            # Synthetic data only  
│   ├─ parameters.py  
│   ├─ forms/  
│   ├─ effects/  
│   ├─ bodies/                           # Uses Haiku – synthetic data only  
│   └─ manifest.py  
├─ indexer/  
│   ├─ tiers/  
│   │   ├─ tier1_qr.py  
│   │   ├─ tier2_policy.py  
│   │   ├─ tier3_fuzzy.py  
│   │   └─ tier4_encoder.py              # Loads fine-tuned model, runs locally  
│   └─ extract/
```

```

|   ├── rules/
|   └── pipeline.py
└── training/                                # NEW in v2 - model training pipeline
    ├── prepare_data.py                    # manifest.parquet → HF Dataset
    ├── train_baseline.py                  # TF-IDF + LogReg
    ├── train_encoder.py                   # Fine-tune XLM-R
    ├── calibrate.py                       # Temperature scaling
    ├── export_onnx.py                     # HF model → ONNX
    ├── benchmark.py                       # Inference latency / throughput
    └── model_card_template.md
└── models/                                # NEW in v2 - trained model artifacts
    ├── baseline-tfidf-v1/
    |   ├── pipeline.pkl
    |   ├── eval_report.html
    |   └── model_card.md
    ├── tessera-encoder-v1/
    |   ├── config.json
    |   ├── pytorch_model.bin (or model.safetensors)
    |   ├── tokenizer files
    |   ├── calibration.json               # Temperature scaling parameter
    |   ├── onnx/model.onnx               # ONNX-exported version
    |   ├── eval_report.html
    |   └── model_card.md
└── eval/
    ├── runner.py
    ├── metrics.py
    ├── compare.py                         # NEW - side-by-side model comparison
    └── reports/
└── dashboard/                             # Next.js
└── data/
    ├── corpus/
    ├── manifest.parquet
    ├── splits/                           # NEW - train/val/test JSON files
    └── runs/
└── README.md

```

4. Phase-by-phase plan

Phase 0 — Foundations (Days 1-2)

Goal: Lock taxonomy, schema format, and rule engine.

Acceptance criteria:

- `taxonomy/taxonomy.yaml` written and validated

- 6 schema YAMLs exist (Repurchase, New Business, Maintenance-Client, Maintenance-Contrib, Claim-Death, Claim-Retirement)
- `indexer/rules/engine.py` evaluates schemas correctly with full unit tests
- One operational sign-off on the taxonomy

Dev agent brief: *unchanged from v1 — same taxonomy and schema YAML structure, same rule engine.* See v1 Phase 0 for full spec.

MLOps notes:

- Schemas are versioned (v1, v2) — never edit in place
 - Schema validation in CI: every schema must round-trip through pydantic models
-

Phase 1 — Generator vertical slice (Days 3-7)

Goal: 50 fully synthetic Repurchase emails — bodies + attachments + ground truth — proving the generation pipeline end-to-end.

Acceptance criteria: *unchanged from v1.*

Dev agent brief: *unchanged from v1, with one explicit framing addition:*

```
IMPORTANT compliance note in code comments: this generator uses Claude Haiku 4.5
to produce
synthetic email bodies. It only ever touches FICTIONAL data — invented investor
names, fake
ID numbers, fictional Meridian Wealth Solutions branding. No real customer mail is
ever sent
to any external API at any point in this pipeline. This is data generation, not
classification.
At inference time (indexer/), no external API is ever called.
```

MLOps notes:

- Every parameter dict that touches Haiku is logged with the prompt and response — full audit of how synthetic data was made
- Generation is reproducible: seed the parameter sampler, pin the Haiku model version
- Cost tracking: $\sim R0.10$ per email $\times 3000 = \sim R300$ total

Outputs end of Phase 1:

- 50 fully synthetic Repurchase emails with attachments and ground truth
- Visual spot-check passed
- Compliance audit trail clear: zero customer data anywhere in the pipeline

Phase 2 — Full corpus + eval harness (Days 8-12)

Goal: Scale to 3,000 emails across all 6 forms; build the eval harness used throughout the rest of the project.

Acceptance criteria:

- All 6 form templates implemented
- 3,000 emails generated across realistic distribution
- Eval harness produces: confusion matrix, per-tier accuracy, calibration plot, latency, cost
- Eval harness can compare TWO indexer implementations side-by-side (this is new in v2 — you'll be comparing TF-IDF vs encoder vs Sonnet ceiling)
- Three baseline runs: dummy (returns ground truth, ~100%), random (~25%), and a "Sonnet ceiling" run that uses Claude Sonnet zero-shot just to set the target accuracy

Dev agent brief — eval comparison tool (new in v2):

```
Build /eval/compare.py.
```

```
Function: compare_runs(run_ids: list[str]) -> ComparisonReport
```

Produces an HTML report at `eval/reports/comparisons/{timestamp}/` with:

- Side-by-side accuracy / auto-route / calibration / latency / cost for each model
- Per-class accuracy comparison (which classes does encoder beat TF-IDF on?)
- Per-quality-tier accuracy comparison (does encoder degrade gracefully on phone photos?)
- Disagreement analysis: emails where models disagree, with ground truth
- "Cost of compliance" headline: accuracy delta between Sonnet ceiling and on-prem encoder

This is the table the deck slide will reference. Build it well.

```
CLI: `python -m eval.compare --runs sonnet-baseline tfidf-v1 encoder-v1`
```

MLOps notes:

- Sonnet ceiling run is one-time, expensive (~R900 for 3000 emails at Sonnet rates), and serves as the "what's possible with no compute constraint" benchmark in the deck
- All eval runs immutable, timestamped, with manifest hash + git SHA recorded
- The "compliance cost" framing — *"on-prem encoder is X points below frontier ceiling, in exchange for zero data egress"* — becomes a deck slide

Outputs end of Phase 2:

- 3,000-email corpus
 - Eval harness with comparison capability
 - Three baseline runs (dummy, random, Sonnet ceiling)
 - Sonnet ceiling number as our accuracy target for the encoder
-

Phase 3 — Deterministic tiers + classical baseline (Days 13-17)

Goal: Build Tiers 1, 2, 3 (all deterministic, zero ML) + a TF-IDF baseline classifier as Tier 4a.

Acceptance criteria:

- All deterministic tiers implemented and individually tested
- TF-IDF + Logistic Regression baseline trained, evaluated, and exposed as Tier 4a
- Baseline target: $\geq 80\%$ accuracy when invoked (note: invoked only on the residual 20-30% that deterministic tiers can't resolve)
- Pipeline cascades correctly: tier 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4a
- Baseline pipeline eval run achieves $\geq 55\%$ auto-route at $\geq 95\%$ precision

Dev agent briefs — Tiers 1, 2, 3: *unchanged from v1.*

Dev agent brief — Tier 4a (TF-IDF baseline):

Build `/training/train_baseline.py` and `/indexer/tiers/tier4_encoder.py` (the latter starts as a TF-IDF classifier and gets upgraded in Phase 4).

Why build TF-IDF before fine-tuning the encoder?

1. Sets a fully-explainable accuracy floor. Investors can ask "what does the model look at?"
and you can show feature weights.
2. Validates the eval harness end-to-end before more expensive training runs.
3. Some customers may prefer the classical model – sub-1MB, sub-10ms inference, fully interpretable. Having it as a deployment option opens the smaller customers.
4. Useful as an ensemble check against the encoder – if both agree, very high confidence.

Implementation:

1) `/training/prepare_data.py`

- Load `data/manifest.parquet`
- Concatenate body + first 3000 chars of OCR'd attachment text into single "input_text" field
- Split: 70% train, 15% val, 15% test
- CRITICAL: split by INVESTOR ID, not by email. Multiple emails from the same investor must all land in the same split. Otherwise data leakage.
- Stratify by `sub_type` within each investor
- Save splits as `data/splits/{train,val,test}.parquet`

2) `/training/train_baseline.py`

- Load splits
- Build sklearn Pipeline: `TfidfVectorizer (n-grams 1-2, max_features 20000, sublinear_tf, min_df=2) → LogisticRegression (multinomial, class_weight='balanced', C=1.0)`
- Fit on train, evaluate on val
- Tune: try `max_features` in [10k, 20k, 50k], `C` in [0.1, 1.0, 10.0] via `GridSearchCV`
- Save pipeline to `models/baseline-tfidf-v1/pipeline.pkl`
- Generate `eval_report.html` with: confusion matrix, top features per class, calibration curve
- Generate `model_card.md` with: training data summary, eval results, intended use, limitations

3) `/indexer/tiers/tier4_encoder.py` (initial version)

- Loads `models/baseline-tfidf-v1/pipeline.pkl` on import
- Function: `tier4_encoder(email_body, attachment_text) -> IndexerOutput`
- Concatenates body + attachment_text

- Calls `pipeline.predict_proba()`
- Returns top-1 prediction with probability as confidence
- Reasoning: top 5 features that contributed most to the chosen class
- Tier 4 ALWAYS returns a result (no None cascade – it's the catch-all)

CLI: ``python -m training.train_baseline``

Acceptance test: `pipeline.pkl` loads in <100ms, predicts in <10ms per email

MLOps notes:

- Class imbalance is the silent killer here. Claims will be ~10% of corpus. Use `class_weight='balanced'` in LogReg AND inspect the confusion matrix per class — overall accuracy can be 90% while claims accuracy is 60%.
- Top-features explanation per prediction is genuinely useful at the dashboard level: *"This was classified as 'repurchase_csp' because of the words 'withdraw', 'GIA-', 'partial reinvestment'."*
- Save TF-IDF vocabulary alongside the model — version everything together

Outputs end of Phase 3:

- Deterministic tiers complete
- TF-IDF baseline trained and integrated as Tier 4a
- First end-to-end pipeline eval: probably 55-65% auto-route, 90-93% accuracy at HC

Phase 4 — Fine-tuned encoder (Days 18-23)

Goal: Train, calibrate, and deploy XLM-RoBERTa-base as the production Tier 4 classifier. This is the centerpiece of the v2 roadmap.

Acceptance criteria:

- Fine-tuned XLM-R model trained, achieving $\geq 92\%$ accuracy on test set
- Temperature-scaled calibration: $ECE < 0.05$
- ONNX export validated to produce identical predictions as PyTorch version (within numerical tolerance)
- Inference latency on CPU $\leq 500\text{ms}$ p95
- Tier 4 in pipeline upgraded to use encoder, with TF-IDF retained as fallback
- Full pipeline eval achieves $\geq 75\%$ auto-route at $\geq 95\%$ precision (target) or $\geq 60\%$ (floor)
- Model card written, model artifact bundled for distribution

Dev agent brief — data preparation:

Extend `/training/prepare_data.py` to produce HuggingFace Dataset format.

1. Load `data/splits/{train,val,test}.parquet` (already created in Phase 3)
2. For each split:
 - Build "input_text" by concatenating: subject + "\n\n" + body + "\n\n" + truncated OCR text
 - Truncate concatenated text intelligently: subject always, then first 1500 chars of body, then first 1000 chars of OCR. This fits comfortably in 512 token budget after tokenization.
 - Encode sub_type as integer label using LabelEncoder (save mapping to `label_map.json`)
3. Save as HuggingFace `datasets.Dataset` to `data/splits/hf/{train,val,test}/`

CLI: ``python -m training.prepare_data --format hf``

Dev agent brief — fine-tuning:

Build /training/train_encoder.py.

Environment: Google Colab Free (T4 GPU) OR local Mac M-series via MPS.

Time budget: 4-6 hours of GPU time for a full training run.

requirements.txt for training environment:

```
torch>=2.1
transformers>=4.40
datasets>=2.18
accelerate>=0.28
evaluate>=0.4
scikit-learn>=1.4
pandas>=2.2
pyarrow>=15
```

Training script:

```
import os
import json
import torch
import numpy as np
from datasets import load_from_disk
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer, EarlyStoppingCallback,
)
from sklearn.metrics import accuracy_score, f1_score
from sklearn.utils.class_weight import compute_class_weight

MODEL_NAME = "xlm-roberta-base"
OUTPUT_DIR = "models/tessera-encoder-v1"
NUM_LABELS = 14 # adjust to actual number of sub_types in taxonomy

# Load datasets prepared in prepare_data.py
ds_train = load_from_disk("data/splits/hf/train")
ds_val = load_from_disk("data/splits/hf/val")

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

def tokenize(batch):
    return tokenizer(
        batch["input_text"],
        truncation=True, padding="max_length", max_length=512,
    )

ds_train = ds_train.map(tokenize, batched=True)
```

```

ds_val    = ds_val.map(tokenize, batched=True)
ds_train.set_format("torch", columns=["input_ids","attention_mask","label"])
ds_val.set_format("torch", columns=["input_ids","attention_mask","label"])

# Compute class weights for imbalance handling
labels_train = np.array(ds_train["label"])
class_weights = compute_class_weight(
    "balanced", classes=np.arange(NUM_LABELS), y=labels_train,
)
class_weights = torch.tensor(class_weights, dtype=torch.float32)

# Model with custom loss for class weighting
class WeightedTrainer(Trainer):
    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)
        logits = outputs.logits
        loss_fct =
torch.nn.CrossEntropyLoss(weight=class_weights.to(logits.device))
        loss = loss_fct(logits, labels)
        return (loss, outputs) if return_outputs else loss

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=NUM_LABELS,
)

args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    lr_scheduler_type="cosine",
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="f1_macro",
    greater_is_better=True,
    logging_steps=50,
    fp16=torch.cuda.is_available(),      # MPS users: fp16=False, bf16=True if M3+
    report_to="none",
    seed=42,
)

def compute_metrics(eval_pred):

```

```

logits, labels = eval_pred
preds = np.argmax(logits, axis=-1)
return {
    "accuracy": accuracy_score(labels, preds),
    "f1_macro": f1_score(labels, preds, average="macro"),
    "f1_weighted": f1_score(labels, preds, average="weighted"),
}

trainer = WeightedTrainer(
    model=model, args=args,
    train_dataset=ds_train, eval_dataset=ds_val,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)],
)

trainer.train()
trainer.save_model(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)

# Save label map for inference
with open(os.path.join(OUTPUT_DIR, "label_map.json"), "w") as f:
    json.dump({"id2label": {str(i): l for i, l in enumerate(LABELS)},
               "label2id": {l: i for i, l in enumerate(LABELS)}} , f, indent=2)

Acceptance test:
- Training completes within 6 hours
- Best val f1_macro  $\geq$  0.92
- Best val accuracy  $\geq$  0.93
- No class has accuracy below 0.85 (check per-class breakdown)
- If any class fails: review whether it's underrepresented in training data;
  consider oversampling that class in prepare_data.py and retraining.

```

Dev agent brief — calibration:

Build /training/calibrate.py.

Why calibrate: a model that says "0.95 confidence" should be right 95% of the time. Out of the box, neural network softmax outputs are usually overconfident. Temperature scaling fixes this with a single learned parameter.

Implementation:

```
import torch
import torch.nn as nn
import torch.optim as optim
from datasets import load_from_disk
from transformers import AutoTokenizer, AutoModelForSequenceClassification

MODEL_DIR = "models/tessera-encoder-v1"

# Load model + val set
tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)
model.eval()

ds_val = load_from_disk("data/splits/hf/val")
ds_val = ds_val.map(lambda b: tokenizer(b["input_text"], truncation=True,
                                     padding="max_length", max_length=512),
                   batched=True)
ds_val.set_format("torch", columns=["input_ids", "attention_mask", "label"])

# Collect logits + labels on val set (no gradient)
all_logits, all_labels = [], []
loader = torch.utils.data.DataLoader(ds_val, batch_size=32)
with torch.no_grad():
    for batch in loader:
        out = model(input_ids=batch["input_ids"],
                    attention_mask=batch["attention_mask"])
        all_logits.append(out.logits)
        all_labels.append(batch["label"])
logits = torch.cat(all_logits)
labels = torch.cat(all_labels)

# Optimize temperature with LBFGS
temperature = nn.Parameter(torch.ones(1) * 1.5)
nll = nn.CrossEntropyLoss()
optimizer = optim.LBFGS([temperature], lr=0.01, max_iter=50)
```

```

def closure():
    optimizer.zero_grad()
    loss = nll(logits / temperature, labels)
    loss.backward()
    return loss

optimizer.step(closure)
T = temperature.item()

# Save calibration parameter
import json
with open(f"{MODEL_DIR}/calibration.json", "w") as f:
    json.dump({"temperature": T, "method": "temperature_scaling"}, f, indent=2)

# Compute Expected Calibration Error before vs after
def ece(probs, labels, n_bins=10):
    confidences, predictions = probs.max(dim=1)
    accuracies = (predictions == labels).float()
    ece_val = 0.0
    bin_edges = torch.linspace(0, 1, n_bins + 1)
    for lo, hi in zip(bin_edges[:-1], bin_edges[1:]):
        in_bin = (confidences > lo) & (confidences <= hi)
        if in_bin.sum() > 0:
            avg_conf = confidences[in_bin].mean()
            avg_acc = accuracies[in_bin].mean()
            ece_val += (in_bin.float().mean() * (avg_conf - avg_acc).abs())
    return ece_val.item()

probs_uncal = torch.softmax(logits, dim=1)
probs_cal = torch.softmax(logits / T, dim=1)
print(f"ECE before calibration: {ece(probs_uncal, labels):.4f}")
print(f"ECE after calibration: {ece(probs_cal, labels):.4f}")
print(f"Optimal temperature: {T:.3f}")

Acceptance test:
- Temperature converges to a value typically between 1.2 and 2.5
- ECE after calibration < 0.05
- Reliability diagram saved as PNG to model dir

```

Dev agent brief — ONNX export:

Build /training/export_onnx.py.

Why ONNX: 2-3x faster CPU inference, smaller dependency footprint at deployment time

(customers run ONNX Runtime instead of full PyTorch). Critical for sub-500ms latency target.

Implementation:

```
from pathlib import Path
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification

MODEL_DIR = Path("models/tessera-encoder-v1")
ONNX_PATH = MODEL_DIR / "onnx" / "model.onnx"
ONNX_PATH.parent.mkdir(parents=True, exist_ok=True)

tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_DIR)
model.eval()

# Dummy input for tracing
dummy = tokenizer("placeholder", padding="max_length", max_length=512,
                  truncation=True, return_tensors="pt")

torch.onnx.export(
    model,
    (dummy["input_ids"], dummy["attention_mask"]),
    ONNX_PATH,
    input_names=["input_ids", "attention_mask"],
    output_names=["logits"],
    dynamic_axes={
        "input_ids": {0: "batch"},
        "attention_mask": {0: "batch"},
        "logits": {0: "batch"},
    },
    opset_version=17,
)

# Validate parity
import onnxruntime as ort
import numpy as np

sess = ort.InferenceSession(str(ONNX_PATH), providers=["CPUExecutionProvider"])

with torch.no_grad():
```



```
pt_logits = model(**dummy).logits.numpy()

ort_logits = sess.run(None, {
    "input_ids": dummy["input_ids"].numpy(),
    "attention_mask": dummy["attention_mask"].numpy(),
})[0]

max_diff = np.max(np.abs(pt_logits - ort_logits))
print(f"Max logit diff PT vs ONNX: {max_diff:.6f}")
assert max_diff < 1e-3, "ONNX export drifted – check opset / fp precision"
```

Acceptance test:

- ONNX file exists and loads in ONNX Runtime
- Predictions match PyTorch within 1e-3 logit tolerance
- File size <1.5GB (XLM-R base ONNX is ~1.1GB)
- Inference latency p95 ≤500ms on CPU (benchmark with 100 random emails)

Dev agent brief — Tier 4 production wrapper:

Replace /indexer/tiers/tier4_encoder.py with the production version.

```
import json
import numpy as np
import onnxruntime as ort
from pathlib import Path
from transformers import AutoTokenizer

MODEL_DIR = Path("models/tessera-encoder-v1")

# Singletons loaded once on import
_tokenizer = AutoTokenizer.from_pretrained(MODEL_DIR)
_session = ort.InferenceSession(
    str(MODEL_DIR / "onnx" / "model.onnx"),
    providers=["CPUExecutionProvider"],
)
with open(MODEL_DIR / "label_map.json") as f:
    _label_map = json.load(f)
with open(MODEL_DIR / "calibration.json") as f:
    _T = json.load(f)["temperature"]

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def tier4_encoder(email_body: str, attachment_text: str) -> IndexerOutput:
    text = (email_body or "")[:5000] + "\n\n" + (attachment_text or "")[:3000]
    enc = _tokenizer(text, padding="max_length", max_length=512,
                     truncation=True, return_tensors="np")
    logits = _session.run(None, {
        "input_ids": enc["input_ids"].astype(np.int64),
        "attention_mask": enc["attention_mask"].astype(np.int64),
    })[0]
    # Apply temperature scaling for calibrated confidence
    probs = softmax(logits / _T, axis=-1)[0]
    pred_id = int(np.argmax(probs))
    confidence = float(probs[pred_id])
    sub_type = _label_map["id2label"][str(pred_id)]
    main_type = TAXONOMY.sub_to_main(sub_type) # lookup from taxonomy.yaml

    # Reasoning: top 2 candidate classes with their probabilities
    top2_ids = np.argsort(probs)[::-1][:2]
    reasoning = (
        f"Encoder confidence: {sub_type}={probs[top2_ids[0]]:.2f}, "
        f"{_label_map['id2label'][str(top2_ids[1])]}={probs[top2_ids[1]]:.2f}"
    )
```

```

)

return IndexerOutput(
    main_type=main_type,
    sub_type=sub_type,
    tier=4,
    confidence=confidence,
    reasoning=reasoning,
    audit_trail=[{
        "tier": 4, "model": "tessera-encoder-v1",
        "calibration_T": _T,
        "top_probs": {_label_map["id2label"][str(i)]: float(probs[i])
                       for i in top2_ids},
    }],
    latency_ms=...,      # measured around the run() call
    cost_zar=0.0,        # zero – no API call
)

```

Acceptance test:

- Tier 4 inference p95 ≤500ms on CPU
- Predictions match calibrated confidence (re-run calibration validation)
- cost_zar is always 0.0 (no API call)
- Audit trail includes model version, temperature, top probabilities

Dev agent brief — model card:

Build `/training/model_card_template.md` and instantiate at `models/tessera-encoder-v1/model_card.md`.

Required sections:

- Model details (architecture, base model, fine-tuning method, parameter count)
- Training data (synthetic corpus version, distribution, generation method, key disclaimer
 - that the model was trained ENTIRELY on synthetic data)
- Evaluation results (per-class accuracy, calibration metrics, comparison to TF-IDF baseline
 - and to Sonnet ceiling)
- Intended use (insurance mail indexing, English + Afrikaans, SA market)
- Out-of-scope use (anything other than mail indexing; non-insurance domains; low-resource
 - languages other than Afrikaans/English)
- Limitations and biases
 - Trained on synthetic data: real-world performance may differ; field validation required
 - Synthetic data was generated by Claude Haiku – model may inherit biases from that generator
 - Class imbalance: rare claims types may have lower accuracy
 - Compliance posture (where the model runs, what data leaves the perimeter, audit trail)
- Versioning and changelog

MLOps notes for Phase 4:

- Reproducible training: pin all seeds (torch, numpy, python random), pin transformers version, log full config snapshot to model dir
- Save not just the best checkpoint but also the last 2 — useful if best overfits in subtle ways
- Don't tune training config too aggressively. Defaults (`lr=2e-5`, 5 epochs, batch 16, warmup 0.1) work for 95% of XLM-R fine-tuning tasks. Time spent tuning is usually better spent on data quality.
- Watch for catastrophic class collapse: if loss drops fast but `f1_macro` plateaus, the model might be predicting only the majority class. Per-class metrics on each eval epoch.
- The Mac M-series MPS path: `(device='mps', bf16=True)` (M3+), `(fp16=False)`. Slower than T4 but works overnight.

Outputs end of Phase 4:

- Fine-tuned XLM-R model at `models/tessera-encoder-v1/`
- Calibration applied, `ECE < 0.05`
- ONNX export validated

- Tier 4 in production pipeline upgraded to encoder
 - Model card published
 - Comparison report: TF-IDF vs encoder vs Sonnet ceiling
-

Phase 5 — Tuning, ablation, and dashboard (Days 24-27)

Goal: Drive end-to-end pipeline accuracy to demo targets via failure analysis. Build the demo dashboard with data sovereignty as a featured view.

Acceptance criteria:

- End-to-end pipeline eval: $\geq 75\%$ auto-route at $\geq 95\%$ precision (target) or $\geq 60\%$ (floor)
- Calibration validated end-to-end (each tier's confidence is honest)
- Ablation table built for the deck: deterministic-only / +TF-IDF / +Encoder / +Sonnet ceiling
- Dashboard deployed publicly with data sovereignty view prominent

Dev agent brief — tuning loop:

Daily loop until end of phase:

1. Run full eval against pipeline
2. Open report; review "10 worst cases" by tier
3. Diagnose: deterministic tier failure (regex / schema), or model failure (encoder misclassifying)?
4. Make ONE targeted change:
 - Schema bbox wrong → fix YAML
 - Regex too loose → tighten policy-number registry
 - Encoder systematically wrong on one sub_type → augment training data for that class, retrain (overnight), re-eval
 - Confidence threshold for auto-route wrong → adjust thresholds in pipeline.py
5. Re-eval, diff, keep or revert.

DO NOT:

- Add new tiers or new features
- Refactor for elegance
- Retrain the encoder more than once per day (training takes hours)
- Touch the body generator (synthetic data is locked after Phase 2)

If encoder retraining is needed:

- Generate 200-500 additional synthetic emails focused on the failing class
- Add to existing training split (NOT val or test)
- Retrain from scratch (not continued training) for a clean reproducible artifact
- Re-calibrate, re-export ONNX
- Bump model version: tessera-encoder-v1 → tessera-encoder-v2

Track all changes in /eval/CHANGELOG.md.

Dev agent brief — dashboard with data sovereignty view:

Build /dashboard/ as a Next.js 15 app. Architecture per v1 (FastAPI backend streaming via SSE, shadcn frontend, replay mode, killer demo moments).

CHANGES from v1:

1) Add a FIFTH route: /sovereignty (this becomes a primary deck moment)

Layout:

Top banner: "Tessera processes mail entirely within your infrastructure"

Live counters (updating every second):

[Mails processed today: 1,247]

[External API calls today: 0]

[Customer data sent to third parties: 0 bytes]

[Average inference latency: 187ms]

Architecture diagram (SVG, animated):

Email arrives → Tier 1 (your CPU) → Tier 2 (your CPU) → Tier 3 (your CPU) → Tier 4 (your CPU, fine-tuned model) → Routing decision → Audit log

Highlighted: "Nothing leaves the dotted line."

Compliance posture cards:

- POPIA: data residency satisfied (all processing on-prem)
- Auditability: full per-email trace, deterministic where possible
- Model provenance: trained on synthetic data, no real customer data
- Updates: model artifacts shipped via Docker image, you choose when to

deploy

2) On every other route, add a small persistent badge in the corner:

"On-prem mode" with a green dot, click → opens /sovereignty

3) On the audit drill-down (route 3 in v1), add to the audit trail panel:

- Tier 4 entry shows: "Model: tessera-encoder-v1 (local)"
- "External API calls during this prediction: 0"

4) On the stats route, add a panel showing the comparison table from Phase 4:

On-prem encoder: 75% auto-route, 96% precision, 0 external calls, R0/email

Frontier ceiling (reference only): 78% auto-route, 97% precision, R0.30/email

API

Delta: -3 points accuracy in exchange for full data sovereignty

Killer demo moments – keep the v1 ones, add a third:

Moment 3 (around 5:30 into demo): switch to /sovereignty view, point at the "External API calls today: 0" counter, leave it visible while explaining the compliance story. This is the "I just sold one" moment.

Deploy: Vercel (frontend), Cloud Run (replay backend that itself calls the on-prem indexer).

The Cloud Run backend is for demo convenience only – make this clear in the deck. The actual indexer code is the same code customers would deploy in their own DC.

MLOps notes:

- Cache all predictions before the demo. Replay streams cached results. No live inference during the pitch.
- The /sovereignty view's counters MUST be honest — wire them to actual logs, not mocks. If you're caught with mock numbers in a demo, the trust collapses.
- Have a 60-second backup video of the full demo on your laptop in case Vercel hiccups

Outputs end of Phase 5:

- Demo-ready accuracy
 - Calibration validated
 - Dashboard with /sovereignty view deployed
 - Both killer moments rehearsed
 - Backup video recorded
-

Phase 6 — Pitch prep (Days 28–30)

Goal: 12-slide deck + 8-minute live demo, rehearsed cold.

Demo structure (8 min):

- 0:00–1:30 — The problem. Mail triage as universal back-office tax. Anchor on one customer's annual cost. **Add: the compliance angle — "and they can't use cloud AI for it because POPIA."**
- 1:30–5:00 — Live demo. Mails flowing on /live. First killer moment (legacy form, tier 2 catches policy number). Drill into one audit trail showing tier-1 deterministic path.
- 5:00–6:00 — **Switch to /sovereignty view. Point at zero-API-call counter. Explain on-prem architecture. This is the new selling moment.**
- 6:00–7:00 — Numbers slide. Auto-route %, accuracy, calibration. Comparison table: on-prem vs frontier ceiling, framed as "we chose compliance over 3 accuracy points."
- 7:00–8:00 — Vision. Indexing wedge → expansion path → AI workforce for regulated back-office.

Deck (12 slides):

1. Title — Tessaera, AI workforce for regulated back-office
 2. Problem — every regulated business has manual mail triage AND can't use cloud AI for it
 3. Market — SA insurance + global TAM, with regulated industries explicitly called out
 4. Wedge — start at the inbox, on-prem
 5. Demo screenshots — /live and /sovereignty side by side
 6. Architecture — four tiers, three deterministic, one local model
 7. Methodology — synthetic corpus, fine-tuned XLM-R, calibration
 8. Results — auto-route %, accuracy, calibration. Bar chart: deterministic / +TF-IDF / +encoder / +Sonnet ceiling
 9. Compliance posture — what runs where, what leaves the perimeter (nothing)
 10. Unit economics — cost per email (R0 inference), FTE replaced per insurer, margin model
 11. Expansion path + team — your dissertation framing as governance credibility
 12. Ask — round size, use of funds, milestones
-

5. Risk register

Risk	Likelihood	Impact	Mitigation
Encoder doesn't hit 92% accuracy on synthetic data	Medium	High	TF-IDF baseline as fallback; can also try mDeBERTa-v3 or ensemble
Investor objects "synthetic data isn't real"	High	High	Already in narrative — Month 2 priority is paid pilot with real data
Inference latency >500ms on CPU breaks the live demo	Low	Medium	ONNX export + benchmark in Phase 4; pre-cache predictions for replay
Calibration drifts after retraining	Medium	Medium	Calibration is a Phase 4 acceptance criterion; re-run on every retrain
Class imbalance kills accuracy on claims	Medium	High	Weighted loss in training; per-class metrics on every eval; oversample if needed
Customer demands the +3 points and asks for a frontier model	Low	Low	Offer in-region Vertex AI Claude as a deployment option for customers willing to relax data residency in exchange for accuracy
Demo dashboard breaks during pitch	Low	High	Backup video + screenshots; predictions pre-cached

Risk	Likelihood	Impact	Mitigation
Model file too large for customer IT	Low	Low	XLM-R base ONNX is 1.1GB; if needed, distill to mDistilBERT (~500MB) or quantize int8 (~280MB)

6. Definition of Done

The project is "demo-ready" when ALL of these are true:

- ☐ 3,000-email synthetic corpus generated with full ground truth
- ☐ All 4 indexer tiers implemented and tested
- ☐ Fine-tuned XLM-R model trained, calibrated, and ONNX-exported
- ☐ End-to-end pipeline eval ≥60% auto-route at ≥95% precision (floor)
- ☐ Inference latency ≤500ms p95 on CPU
- ☐ **Zero external API calls at inference time (validated by network monitor during eval)**
- ☐ Dashboard deployed publicly with /sovereignty view
- ☐ Both killer demo moments fire reliably
- ☐ 60-second backup video recorded
- ☐ Comparison table built: deterministic / +TF-IDF / +encoder / +frontier ceiling
- ☐ Model card published, including limitations and synthetic-data disclaimer
- ☐ 12-slide deck reviewed by one outside reader
- ☐ Demo rehearsed cold 10× under 8 min
- ☐ Investor pitch booked

7. Daily tracking template

Day {n} – {date}

Phase: {0|1|2|3|4|5|6}

Hours worked: {h}

Done today:
 -

Blocked on:
 -

Tomorrow:
 -

Eval delta (if applicable):

Auto-route: {prev}% → {new}% ({+/- delta})

HC accuracy: {prev}% → {new}% ({+/- delta})

Calibration ECE: {prev} → {new}

Inference p95 latency: {prev}ms → {new}ms

Notes:

8. What the dev agent should NOT do

1. **Don't call any external API at inference time.** Every external call in `/indexer/` is a bug. Audit by grepping for `anthropic.`, `openai.`, `requests.`, `httpx.` inside the indexer module.
2. **Don't generate more synthetic data to fix accuracy problems** unless a specific class is genuinely underrepresented. Usually data quality > quantity.
3. **Don't add features.** Closed scope: 6 forms, 4 tiers, 1 production model, 1 baseline model, 1 dashboard.
4. **Don't tune the body generator (Haiku) and the encoder at the same time.** Lock body generation at end of Phase 2.
5. **Don't trust softmax confidence without calibration.** Phase 4 calibration is non-negotiable.
6. **Don't deploy the encoder to PyTorch in production.** Always go through ONNX — 2-3x speedup matters for the latency target and for the model card story.
7. **Don't use real customer data anywhere.** Even hypothetically, even in tests, even if a friend at Sanlam offers. The compliance story collapses the moment a real ID number ever touches your training set.
8. **Don't deploy on demo day.** Final deploy 24h before pitch, full rehearsal on deployed environment.

Appendix A — what you tell the CISO

(Draft response template for when you pitch a real customer.)

Tessera ships as a Docker image. It runs in your infrastructure. Customer mail never leaves your perimeter. The model — a fine-tuned XLM-RoBERTa classifier — was trained entirely on synthetic data we generated ourselves; no real customer data from any party was used at any stage of model training. At inference time, the system makes zero external API calls; we'll demonstrate this with a network monitor of your choice during deployment validation. Every prediction has a full audit trail: which tier resolved it, what fields were extracted, what rule

matched, what the model's confidence was. Model updates are shipped as new Docker image versions; you control when they're deployed. The audit log is your audit log, in your database. POPIA cross-border transfer concerns do not arise because no transfer occurs.

If they push for the extra accuracy of a frontier model: we offer in-region Vertex AI Claude as an optional deployment, with a separate DPA. But that's their choice — the default is fully on-prem.

Appendix B — what to put in the README of the customer-facing repo

When you deliver the eventual pilot to Sanlam, the README must include:

- One-paragraph what-this-is
 - Hardware requirements: 8 CPU cores, 16GB RAM, 5GB disk for model + corpus
 - `docker pull tessera/indexer:v1` + run command
 - A single env-var-driven config file
 - A self-test command: `tessera-indexer self-test` runs a small bundled test corpus, prints accuracy + verifies zero external calls
 - The model card link
 - The audit log schema documentation
 - The escalation contract: which mails always go to human review regardless of model output
 - A "model update" runbook: how to update to v2 when shipped
-

End of v2 roadmap. Update the daily tracking template, commit it to /docs/journal/, and start Phase 0 today.